Chapitre7: Les threads

Définition

Un **thread**, ou **processus léger**, est l'entité constituée par un programme en cours d'exécution. Cela se matérialise par un couple (**code**, **données**) : le code en cours d'exécution et les données que ce code traite.

- Il est capable de s'exécuter en **parallèle** à d'autres traitements.
- Le principal avantage des threads est de pouvoir répartir différents traitements d'un même programme en plusieurs unités distinctes pour permettre leurs exécutions "**simultanées**".

Définition

- Sur une machine monoprocesseur, c'est le système d'exploitation qui alloue du temps d'utilisation du CPU pour accomplir les traitements de chaque threads à "tour de rôle" (un **parallélisme simulé**).
- Sur une machine multiprocesseur, le système d'exploitation peut répartir l'exécution sur plusieurs cœurs (un **parallélisme vrai**).

Création du thread

- Les threads peuvent être créés comme instance d'une classe dérivée de la classe **Thread** ou de l'interface **Runnable**.
- Elles sont lancées par la méthode **start()**, qui demande à l'ordonnanceur de thread de lancer la méthode **run()** du thread.
- Cette méthode **run()** doit être implantée dans le programme.

Syntaxe:

```
public class SimpleThread extends Thread {
      public void run() {
            //code exécuté dans un processus propre
      }
}
```

```
public class SimpleThread implements Runnable{
      public void run() {
            //code exécuté dans un processus propre
      }
}
```

Appel:

```
SimpleThread unSimpleThread = new SimpleThread();
unSimpleThread.start();
```

Méthode 1

```
SimpleThread unSimpleThread = new SimpleThread();
Thread thread = new Thread(unSimpleThread);
thread.start();
```

Méthode 2

Exemple: (méthode 1)

Un thread 5 fois affiche son nom et un entier croissant, puis s'endort pour une période comprise entre 0 et 1000 millisecondes:

```
public class ThreadExp1 extends Thread{
    public ThreadExp1(String nom) {
        super(nom);
    }
    public void run() {
        for(int i=0;i<5;i++){
            System.out.println(getName()+" "+i);
            try {
                sleep((long)(Math.random()*1000));
            } catch (InterruptedException e) {e.printStackTrace();}
        }
        System.out.println(getName()+" terminé");
    }
}
```

Exemple: (Méthode 1)

La classe principale

```
public class App1 {
 public static void main(String[] args) {
  ThreadExp1 t1=new ThreadExp1("DSI");
  ThreadExp1 t2=new ThreadExp1("SE");
  t1.start();
  t2.start();
 }
}
```

```
public class App1 {
public static void main(String[] args) {
ThreadExp1 t1=new ThreadExp1("DSI");
t1.start();
for(int i=0;i<5;i++){
System.out.println("Main "+i);
try {
Thread.sleep((long)(Math.random()*1000));
} catch (InterruptedException e) {
e.printStackTrace();
}
}
System.out.println("Main terminé");
}
}
```

Résultat de l'affichage

```
SE 0
DSI 0
SE 1
DSI 1
SE 2
SE 3
SE 4
DSI 2
SE terminé
DSI 3
DSI 4
DSI terminé
```

Remarque:
Le programme **main** est aussi un thread

Exemple: (Méthode 2)

```
public class ThreadExp3 implements Runnable{
 private String nom;
  public ThreadExp3(String nom) {
  this.nom=nom;
 }
 public void run() {
  for(int i=0;i<5;i++){
   System.out.println(nom+" "+i);

   try {
    Thread.sleep((long)(Math.random()*1000));
   } catch (InterruptedException e) {
    e.printStackTrace();
   }
  }
  System.out.println(nom+" terminé");
 }
}
```

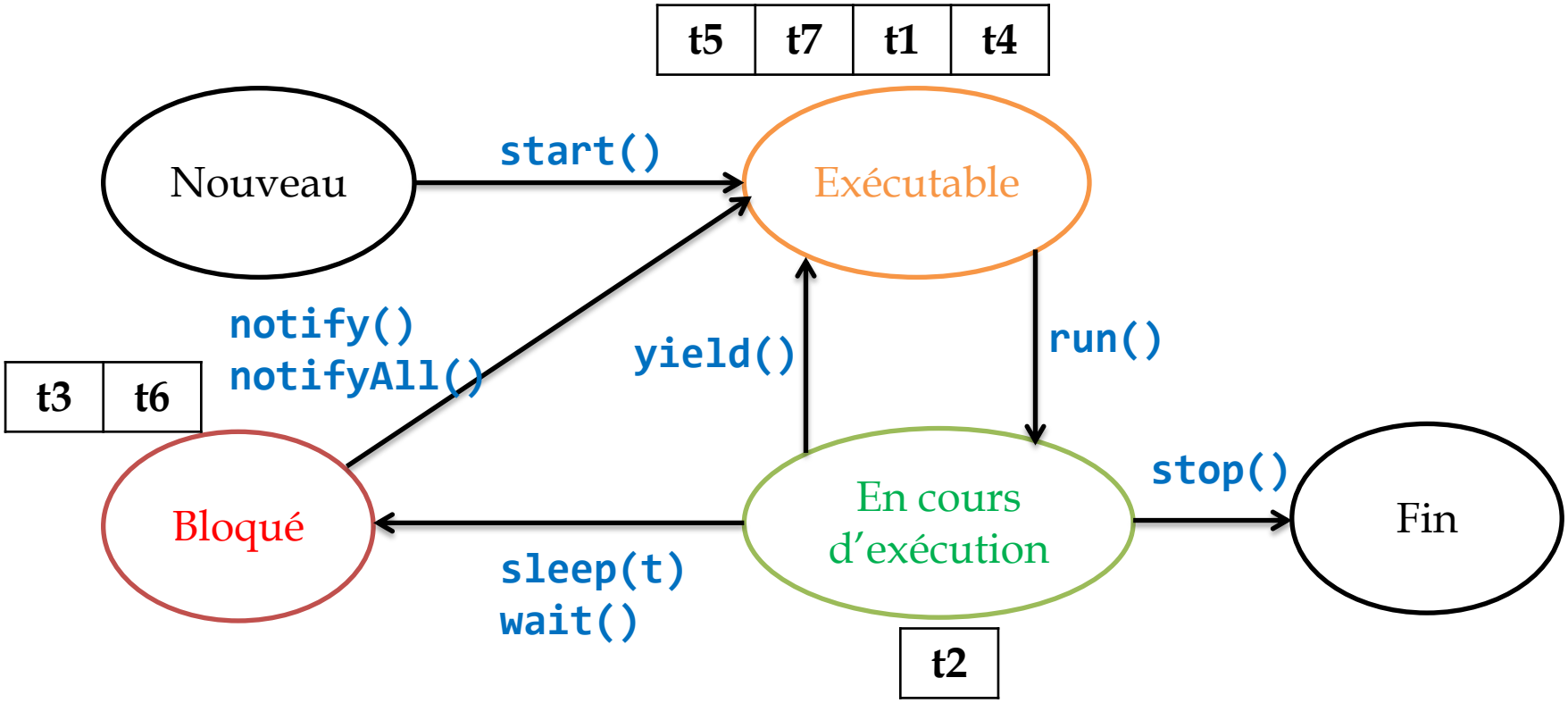
```
public class Test {
 public static void main(String[] args) {
 Thread t1=new Thread(new ThreadExp3("DSI"));
 Thread t2=new Thread(new ThreadExp3("SE"));
 t1.start();
 t2.start();
 }
}
```

Exemple: (Méthode 3)

```
public class ThreadExp4 implements Runnable{
 private String nom;
 private Thread t=new Thread(this);
 public ThreadExp4(String nom) {
  this.nom=nom;
  t.start();
 }
 public void run() {
  for(int i=0;i<5;i++){
   System.out.println(nom+" "+i);
   try {
    Thread.sleep((long)(Math.random()*1000));
   } catch (InterruptedException e) {
    e.printStackTrace();
   }
 }
 System.out.println(nom+" terminé");
 }
}
```

```
public class Test2 {
 public static void main(String[] args) {
  ThreadExp4 t1=new ThreadExp4("DSI");
  ThreadExp4 t2=new ThreadExp4("SE");
 }
}
```

Cycle de vie d'un thread



Un seul processus ici

Synchronisation

Dans un programme, la partie à partir de laquelle on accède à une ressource partagée se nomme **section critique**. Pour éviter les problèmes d'accès concurrent, il faut empêcher que deux processus se retrouvent simultanément dans leurs sections critiques. **Exemple**: (Accès simultané à un compte pour effectuer un **retrait**)

```
public class Compte {
 private float solde;
 public Compte(float solde) {
  this.solde=solde;
 }
 public float getSolde() {
  return solde;
 }
 public boolean retirer(float montant) {
  if(montant<=solde){
   solde=solde-montant;
   return true;
  }
  return false;
 }
}
```

```
public class Client extends Thread{
 private String nom;
 private Compte compte;
 private float mt;
 public Client(String nom, Compte compte,
                    float mt) {
  this.nom=nom;
  this.compte=compte;
  this.mt=mt;
 }
 public void run() {
  System.out.println("Solde
          "+compte.getSolde()+" lu par "+nom);
  boolean oui=compte.retirer(mt);
  if(oui)System.out.println(compte.getSolde()
          +" retrait effectué par:"+nom);
 }
}
```

Synchronisation

On constate ici le problème d'accès concurrent, ce qui donne des résultats erronés (le solde doit être 500 au lieu de 800)

```
public class TestCompte {
 public static void main(String[] args) {
  Compte cpt=new Compte(1000);
  Client cl1=new Client("Ali", cpt,200);
  Client cl2=new Client("Rachid", cpt, 300);
  cl1.start();
  cl2.start();
 }
}
```

Résultat

```
Solde 1000.0 lu par Ali
Solde 1000.0 lu par Rachid
500.0 retrait effectué par:Rachid
800.0 retrait effectué par:Ali
```

Synchronisation

Pour résoudre le problème, on synchronise le bloc de la section critique en utilisant le modificateur **synchronized** sur la méthode ou le bloc en question (ici il s'agit d'un bloc d'instructions).

```
public class Client extends Thread{
 //…
 public void run() {
  synchronized (compte) {
   System.out.println("Solde "+compte.getSolde()+"
          lu par "+nom);
   boolean oui=compte.retirer(mt);
  if(oui)System.out.println(compte.getSolde()
          +" retrait effectué par:"+nom);
  }
 }
}
```

Résultat

```
Solde 1000.0 lu par Rachid
700.0 retrait effectué par:Rachid
Solde 700.0 lu par Ali
500.0 retrait effectué par:Ali
```

Ici on synchronise la méthode **retirer**

```
public synchronized boolean  retirer(float montant) {
 if(montant<=solde){
  solde=solde-montant;
  return true;
 }
 return false;
 }
}
```

Synchronisation par l'envoie de message

La classe **Object** contient les méthodes `wait()`, `notify()` et `notifyAll()` pour permettre de **synchroniser** des threads grâce à l'envoi de messages. Ces méthodes permettent la mise en œuvre d'un mécanisme de communication par échanges de messages visant à synchroniser l'exécution de threads.

Exemple: Producteur - Consommateur

Considérons la situation suivante :

- Un certain nombre de fois (par exemple 10) chaque producteur «fabrique» un produit (numéro), le dépose dans un entrepôt qui ne peut en contenir qu'un, puis dort un temps variable.
- Un certain nombre de fois (par exemple 10 encore) chaque consommateur prend le produit qui se trouve dans l'entrepôt et le «consomme».

Synchronisation par l'envoie de message

Entrepot

```
public class Entrepot {
    private int contenu;
    public void prendre(String nom){
        System.out.println("Le consommateur "+nom+" obtient "
        +contenu);
    }
    public void deposer(String nom, int valeur){
        contenu=valeur;
        System.out.println("Le producteur "+nom+" produit " +contenu);
    }
}
```

Synchronisation par l'envoie de message

Producteur

```
public class Producteur extends Thread{
    private Entrepot entrepot;
    private String nom;
    public Producteur(Entrepot entrepot, String nom) {
        this.entrepot=entrepot;
        this.nom=nom;
    }
    public void run() {
        for(int i=0;i<10;i++){
            entrepot.deposer(nom,i);
        try {
            sleep((long)(Math.random() * 100));
        }catch (InterruptedException e) {
            System.out.println(e.getMessage());
        }
        }
    }
}
```

Synchronisation par l'envoie de message

Consommateur

```
public class Consommateur extends Thread{
    private Entrepot entrepot;
    private String nom;
    public Consommateur(Entrepot entrepot, String nom) {
        this.entrepot=entrepot;
        this.nom=nom;
    }
    public void run() {
        for(int i=0;i<10;i++){
            entrepot.prendre(nom);
        }
    }
}
```

Synchronisation par l'envoie de message

Test

```
public class TestEntrepot {
    public static void main(String[] args) {
        Entrepot entrepot = new Entrepot();
        Producteur producteur = new Producteur(entrepot, "Ali");
        Consommateur consommateur = new Consommateur(entrepot, "Rachid");
        producteur.start();
        consommateur.start();
    }
}
```

Résultat

```
Le producteur Ali produit 0
Le consommateur Rachid obtient 0
Le consommateur Rachid obtient 0
Le consommateur Rachid obtient 0
…
Le producteur Ali produit 1
Le producteur Ali produit 2
…
Le producteur Ali produit 9
```

Ici un problème: le producteur produit « **0** » alors que consommateur consomme **10 fois** le même produit « **0** » ce qui n'est pas possible puisque une fois le produit est consommé l'entrepôt devient **vide**.

Synchronisation par l'envoie de message

Exemple: Producteur – Consommateur (Diagnostique du problème)

- Les producteurs et les consommateurs **ne se connaissent** pas et ne prennent aucune mesure pour prévenir les **conflits**,
- Lorsque l'entrepôt est **vide** les consommateurs ne peuvent pas consommer et lorsqu'il est **plein** les producteurs ne peuvent pas produire,
- C'est l'entrepôt **qui gère** ces conflits,
- L'entrepôt est **indépendant** du nombre de producteurs et de consommateurs qui traitent avec lui.

Synchronisation par l'envoie de message

Résolution du problème

```
public class Entrepot {
    private int contenu;
    private boolean disponible = false;
    public synchronized void prendre(String nom){
        while(disponible==false){
            try {
                wait();
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
        System.out.println("Le consommateur "+nom+" obtient "
        +contenu);
        disponible=false;
        notifyAll();
    }
```

Le thread « consommateur » est bloqué

Le thread « consommateur » obtient le produit et réveille les autres producteurs

Synchronisation par l'envoie de message

Résolution du problème

```
public synchronized void deposer(String nom, int valeur){
    while(disponible==true){
        try {
            wait();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
    contenu=valeur;
    System.out.println("Le producteur "+nom+" produit "
    +contenu);
    disponible=true;
    notifyAll();
}
}
```

Le thread « producteur » est bloqué

Le thread « producteur » dépose le produit et réveille les autres consommateurs

Synchronisation par l'envoie de message

Résultat obtenu

```
Le producteur Ali produit 0
Le consommateur Rachid obtient 0
Le producteur Ali produit 1
Le consommateur Rachid obtient 1
Le producteur Ali produit 2
Le consommateur Rachid obtient 2
Le producteur Ali produit 3
Le consommateur Rachid obtient 3
Le producteur Ali produit 4
Le consommateur Rachid obtient 4
Le producteur Ali produit 5
Le consommateur Rachid obtient 5
Le producteur Ali produit 6
Le consommateur Rachid obtient 6
Le producteur Ali produit 7
Le consommateur Rachid obtient 7
Le producteur Ali produit 8
Le consommateur Rachid obtient 8
Le producteur Ali produit 9
Le consommateur Rachid obtient 9
```